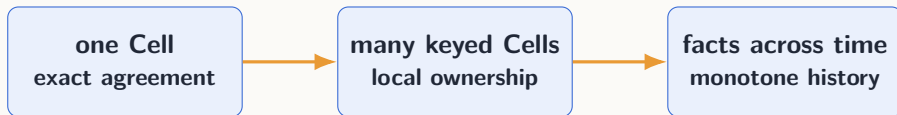

From Cells to Memory

Protocols as Resources in CRIS

A memory map, shared reads, frozen facts, and snapshots of the past

Presenter name ■ 29 July 2026

Where this talk starts



From the preceding Cell example, keep only three ideas:

- ▶ the client owns a **capability** such as $\text{cell}(v)$,
- ▶ the module retains a matching **authoritative view**, and
- ▶ global validity makes their claims compatible.

Today

Scale that pattern from one slot to a memory, then change the compatibility relation from “equal now” to “no smaller than a past observation.”

The promise

By the end, you should be able to:

Read

Read the four memory rules and tell which resource crosses each call boundary.

Predict

Predict what an update changes—and what it preserves automatically.

Choose

Choose a standard resource algebra from the protocol a client needs.

Start from the protocol, not from the algebraic carrier.

Protocols already hide in ordinary programs

Key–value ownership

“I may access key k , and it currently contains v .”

Publication / freeze

“This final result may be copied forever and can no longer change.”

Shared immutable access

“Several clients may read; nobody may write until the shares return.”

Monotonic history

“A later observation cannot be smaller than this past one.”

Thirty-second prompt

Name another real program protocol as: *a capability + an allowed transition*.

Running example: a one-cell allocation

```
p = Mem.alloc(1);  
Mem.store(p, 7);  
x = Mem.load(p);  
Mem.free(p);
```

The concrete module keeps a private heap and implements these operations with internal state reads and writes.

The modularity question

What prevents an unknown component from storing through `p` between our store and load?

“We inspected all its code”
is not a modular answer.

The checked-in implementation's `free` removes one cell, so the main example uses `alloc(1)`.

The client-facing memory contract

Read $\ell \mapsto_{\gamma_m} v$ as: “I own access to ℓ , whose current value is v .”

operation	required resource	returned resource / fact
<code>alloc(n)</code>	valid size	fresh region $\ell + i \mapsto_{\gamma_m} \text{undef}$
<code>load(l)</code>	$\ell \mapsto_{\gamma_m}^q v$	$\text{ret} = v$ and the same fraction
<code>store(l,w)</code>	full $\ell \mapsto_{\gamma_m} v$	full $\ell \mapsto_{\gamma_m} w$
<code>free(l)</code>	full $\ell \mapsto_{\gamma_m} v$	resource consumed

A call boundary transfers capabilities

The precondition tells the module what the caller authorizes; the postcondition tells the caller what capability comes back.

Why not expose one global heap fact?

Global, fragile

$$\text{heap} = M$$

A store anywhere changes the assertion.
Every client must reason about the whole map again.

Local, stable

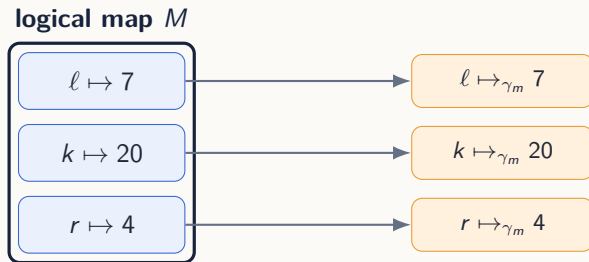
$$\ell \mapsto_{\gamma_m} 7 * k \mapsto_{\gamma_m} 20$$

A store at ℓ should preserve the framed fact about k :

$$\ell \mapsto_{\gamma_m} 8 * k \mapsto_{\gamma_m} 20.$$

Give each client exactly the entry it may rely on.

A memory is a map of Cells



ghost_map packages the repeated Cell pattern

One authoritative view of the whole dictionary; independent points-to-like fragments for keys.

Artifact note: MemA.v manually builds an equivalent map. CRIS also has a compatible interface in CRIS/theories/iris_system/lib/ghost_map.v, but current MemA does not use it.

Two views of one logical map

Module view

$$\text{Auth}_{\gamma_m}(M)$$

- ▶ authoritative view of the whole map,
- ▶ kept at full fraction inside the module proof,
- ▶ ghost state—not the physical heap.

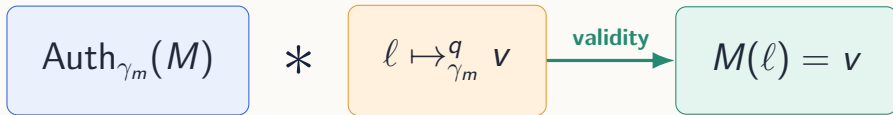
Client view

$$\ell \mapsto_{\gamma_m}^q v$$

- ▶ fragment for one key,
- ▶ carries value knowledge and permission,
- ▶ the ghost name γ_m links both views.

One physical location, one logical entry, two compatible ownership views.

The one law to remember



- ▶ Validity checks whether all simultaneously owned claims can coexist.
- ▶ A client cannot forge $\ell \mapsto_{\gamma_m} w$ when authority records $v \neq w$.
- ▶ For a different key, a fragment imposes a different, independent lookup constraint.

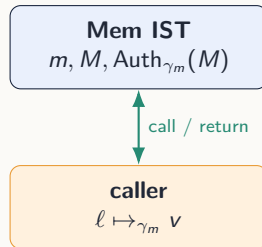
ghost_map_lookup

Where authority lives: the CRIS IST

Simplified module-local invariant

$$\begin{aligned} \text{IST}(st_A, st_I) &\triangleq \exists m, M. st_I[\text{Mem.mem}] = m \\ &\quad * \ulcorner \text{repr}(m) = M \urcorner \\ &\quad * \text{Auth}_{\gamma_m}(M) \end{aligned}$$

At each simulation interaction point, the private concrete heap and the authoritative logical map agree.



Boundary discipline

A method combines hidden authority with the caller's fragment, restores the IST, and returns the promised fragment.

The packaged rules are the protocol

logical operation	resource transition / consequence	needed permission
lookup	$\text{Auth}_{\gamma_m}(M) * \ell \mapsto_{\gamma_m}^q v \Rightarrow M(\ell) = v$	any compatible fractions
insert	$\text{Auth}_{\gamma_m}(M) \rightsquigarrow \text{Auth}_{\gamma_m}(M[\ell := v]) * \ell \mapsto_{\gamma_m} v$	full authority; fresh key
update	$\text{Auth}_{\gamma_m}(M) * \ell \mapsto_{\gamma_m} v \rightsquigarrow \text{Auth}_{\gamma_m}(M[\ell := w]) * \ell \mapsto_{\gamma_m} w$	full authority + full element
delete	$\text{Auth}_{\gamma_m}(M) * \ell \mapsto_{\gamma_m} v \rightsquigarrow \text{Auth}_{\gamma_m}(M \setminus \{\ell\})$	full authority + full element

`ghost_map_lookup` `ghost_map_insert` `ghost_map_update` `ghost_map_delete`

Notice what is absent

Clients never receive the authority. They receive only the fragment needed to authorize one entry's operation.

Allocate: extend the map and create ownership

$$\text{Auth}_{\gamma_m}(M) \rightsquigarrow \text{Auth}_{\gamma_m}(M[\ell := \text{undef}]) * \ell \mapsto_{\gamma_m} \text{undef} \quad \ell \notin \text{dom}(M)$$

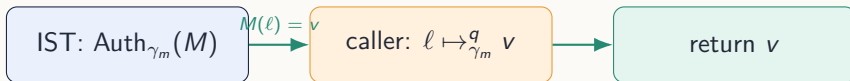
1. The concrete allocator selects a fresh block/location.
2. The proof extends the authoritative logical map at that same key.
3. The new fragment crosses the call boundary to the client.

Ownership is created at allocation time

Freshness is not just a numeric property of the address. It permits allocating a new compatible logical fragment.

`ghost_map_insert` / `ghost_map_insert_big`

Load: agreement without change



authority and fraction are unchanged

- ▶ The fragment identifies the relevant entry and its value.
- ▶ The IST connects that logical entry to the concrete heap lookup.
- ▶ No knowledge of any other location is required.

Read-only steps preserve every compatible frame

This is why any positive owned fraction—and later a persistent read-only fragment—can justify a load.

Store: change exactly one entry

$$\text{Auth}_{\gamma_m}(\{\ell:7, k:20\}) * \ell \mapsto_{\gamma_m} 7 * k \mapsto_{\gamma_m} 20$$

$\rightsquigarrow$

$$\text{Auth}_{\gamma_m}(\{\ell:8, k:20\}) * \ell \mapsto_{\gamma_m} 8 * k \mapsto_{\gamma_m} 20$$

Changed

The authority and full fragment at ℓ move together from 7 to 8.

Framed

The token for k stays compatible and therefore remains untouched.

One store proof, end to end

1. **Open the IST:** obtain concrete heap m , logical map M , and $\text{Auth}_{\gamma_m}(M)$.
2. **Check the entry:** combine authority with $\ell \mapsto_{\gamma_m} v$ to learn $M(\ell) = v$, hence concrete lookup succeeds.
3. **Execute:** step the concrete $S\text{Get}/S\text{Put}$ implementation to heap m' .
4. **Update ghost state:** jointly obtain $\text{Auth}_{\gamma_m}(M[\ell := w]) * \ell \mapsto_{\gamma_m} w$.
5. **Close and return:** prove $\text{repr}(m') = M[\ell := w]$, restore the IST, return the new fragment.



Mechanized instance: `imp_system/mem/MemIAproof.v:325-346`.

Audience check: what must Bob reprove?

Alice

$$\ell \mapsto_{\gamma_m} 7$$

calls `store(1,8)`.

Bob

$$k \mapsto_{\gamma_m} 20$$

where $k \neq \ell$.

Question

Which fact about k must Bob re-establish after Alice's call?

Answer

Nothing. Once $k \neq \ell$ is known, Bob's token is a compatible frame before and after the frame-preserving update.

Fractions turn exclusivity into shared reads

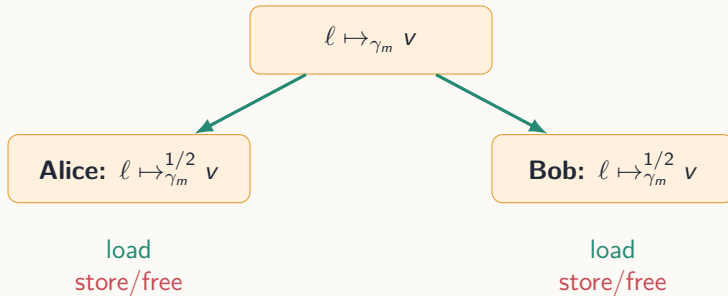
$$\ell \mapsto_{\gamma_m}^{q_1+q_2} v \iff \ell \mapsto_{\gamma_m}^{q_1} v * \ell \mapsto_{\gamma_m}^{q_2} v \quad (q_1 + q_2 \leq 1)$$

- ▶ The fractions are **permission accounting**, not probabilities.
- ▶ Same-key fragments must agree on v .
- ▶ Full ownership means fraction 1; it can be split and later recombined.

`ghost_map_elem_fractional`

`ghost_map_elem_agree`

Two readers, no writer



To write again

Recombine every outstanding share to recover fraction 1, then invoke the full-element update rule.

A Rust analogy—and its boundary

Fractional permission story	Rust intuition
several proper fractions may read	several shared <code>&T</code> borrows
no proper fraction may write	shared aliases exclude mutable access
recombine all shares to fraction 1	all shared borrows have ended
full token authorizes update	unique mutable access

Analogy, not a formal Rust model

`ghost_map` does not itself encode lexical lifetimes, reborrowing, compiler inference, or Rust's full type semantics.

The useful common idea is: **outstanding read shares prevent write authority**.

Why a half-share cannot store

Suppose Alice and Bob each own a half-share asserting v :

$$\ell \mapsto_{\gamma_m}^{1/2} v * \ell \mapsto_{\gamma_m}^{1/2} v.$$

If Alice alone changed her token to $w \neq v$, Bob's compatible frame would remain:

$$\ell \mapsto_{\gamma_m}^{1/2} w * \ell \mapsto_{\gamma_m}^{1/2} v \quad \text{invalid: same key, disagreeing values.}$$

Frame-preserving update test

A legal update must preserve validity with *every* compatible environmental frame. Therefore no writer exists until all shares are recombined.

This matters even in a sequential program: Bob may be another module, callback, or later phase.

DFrac adds persistent read-only knowledge

Owned fraction

$$\ell \mapsto_{\gamma_m}^q v$$

Quantitative, splittable permission. Full fraction 1 is required for update/delete.

Discarded knowledge

$$\ell \mapsto_{\gamma_m}^{\square} v$$

Persistent and freely duplicable. It records a permanently frozen value.

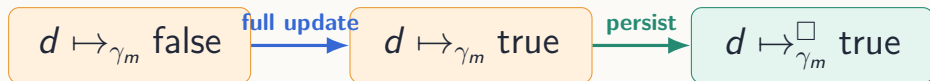
$$\ell \mapsto_{\gamma_m}^q v \rightsquigarrow \ell \mapsto_{\gamma_m}^{\square} v$$

What is discarded?

Discard the *write capability*; retain duplicable knowledge of the value.

`ghost_map_elem_persist`

Completion flag: update first, publish second



- ▶ Before completion, full ownership authorizes the one update.
- ▶ After completion, persistent true can be copied to any number of clients.
- ▶ The persistent frame prevents full write ownership from ever being recovered.

Use case

Publish an initialized table, a completed task result, or finalized configuration as immutable knowledge.

Persistence freezes the value you have *now*

Wrong order

$$d \mapsto_{\gamma_m} \text{false} \rightsquigarrow d \mapsto_{\gamma_m}^{\square} \text{false}$$

This freezes the flag at false. It does *not* reserve a future transition to true.

If observers need facts before the transition

Use a monotone / one-shot state-machine protocol. Encode $\text{false} < \text{true}$, permit only upward movement, and publish a persistent completion witness afterward.

Upstream Iris also provides `ghost_map_elem_unpersist`, but it yields only an unspecified fraction—never the full fraction needed to write while a persistent frame exists. (The current CRIS port leaves this lemma commented out.)

Checkpoint: classify the capability

token	load	store/free	split	copy freely	intended reading
full owned	yes	yes	yes	no	mutable exclusive access
proper fraction	yes	no	yes	no	shared read permission
discarded / persistent	yes	no	—	yes	immutable published fact

Quick retrieval questions

1. Two components must read: which token shape?
2. A component must store: what must be recombined?
3. Everyone may retain the final value forever: which transition?

The standard `ghost_map` API directly supports persistent reads. The checked-in `MemA.load` is currently parameterized by an owned rational fraction.

A benchmark across unknown code

```
before = Mem.get_count();  
f();                                     // arbitrary; may call Mem  
after = Mem.get_count();  
assert(0 <= after - before);           // if f returns
```

Counter policy

Count completed alloc/load/store/free calls. Query itself does not increment.

Proof challenge

After f , we need a stable fact about an *old* observation even though the current state may have changed.

Model the counter as an unbounded natural. A machine implementation must separately rule out wraparound or specify it.

A result-only query spec forgets history

A weak specification says only:

$$\text{get_count}() \quad \{ \exists n. \text{ret} = n * \text{Current}(n) \}.$$

Calling it twice produces unrelated names:

$$\exists n_0. \text{before} = n_0 \qquad \exists n_1. \text{after} = n_1.$$

Missing link

What connects n_0 to n_1 after arbitrary f ? Exact current-state ownership cannot remain exact when the state changes.

Desired receipt

The first query should return a durable token $\text{Snap}_{\gamma_c}(n_0)$: “the counter has reached at least n_0 .”

mono_nat: authority plus lower-bound snapshots

Current authority

$$\text{Auth}_{\gamma_c}(n)$$

The module's exact current logical count.

Past receipt

$$\text{Snap}_{\gamma_c}(m)$$

Persistent claim: the current count is at least m . It does not say the count remains exactly m .



“Snapshot” is presentation terminology; the installed proposition is `mono_nat_lb_own`.

Three rules prove the monotone protocol

snapshot $\text{Auth}_{\gamma_c}(n) \implies \text{Snap}_{\gamma_c}(n)$

increase $\text{Auth}_{\gamma_c}(n) * \lceil n \leq n' \rceil \rightsquigarrow \text{Auth}_{\gamma_c}(n') * \text{Snap}_{\gamma_c}(n')$

validity $\text{Auth}_{\gamma_c}(n) * \text{Snap}_{\gamma_c}(m) \implies \lceil m \leq n \rceil$

`mono_nat_lb_own_get`

`mono_nat_own_update`

`mono_nat_lb_own_valid`

These are the exact upstream Iris names. A CRIS implementation needs a small `CRIS.own` wrapper; this checkout has no local `mono_nat.v` port yet.

Compare the validity relations

`ghost_map`: element and authority agree on an **exact value**.

`mono_nat`: snapshot and authority agree through the **order** $m \leq n$.

Compose both protocols inside one IST

Simplified combined invariant

$$\begin{aligned} \text{IST} \triangleq \exists m, M, c, n. \quad & \ulcorner \text{concrete_heap} = m \wedge \text{repr}(m) = M \urcorner \\ & * \text{Auth}_{\gamma_m}(M) \\ & * \ulcorner \text{concrete_count} = c \wedge c = n \urcorner \\ & * \text{Auth}_{\gamma_c}(n). \end{aligned}$$

Memory protocol

Update only the owned key; frame all other element resources.

Counter protocol

Each counted operation moves n to $n + 1$ before reclosing the IST.

Independent protocols compose with separating conjunction / a product RA.

Query spec: supply a past lower bound

The program has no argument. Its *specification* is parameterized by a prior observation m :

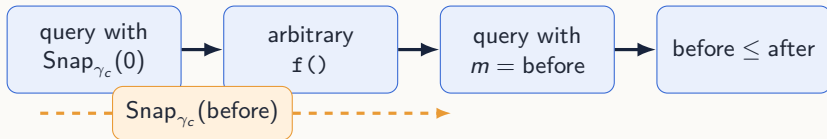
`get_count()` for any ghost parameter m

$$\underbrace{\text{Snap}_{\gamma_c}(m)}_{\text{pre}} \implies \exists n. \underbrace{\lceil \text{ret} = n \wedge m \leq n \rceil * \text{Snap}_{\gamma_c}(n)}_{\text{post}}.$$

1. Open the IST and obtain exact authority $\text{Auth}_{\gamma_c}(n)$.
2. Validity with $\text{Snap}_{\gamma_c}(m)$ gives $m \leq n$.
3. Snapshot the current n , return the concrete value, and reclose the IST.

Because $\text{Snap}_{\gamma_c}(m)$ is persistent, the caller may retain and reuse it. Initialization supplies $\text{Snap}_{\gamma_c}(0)$.

The complete client proof



1. First query returns before and a persistent snapshot of it.
2. Carry the snapshot across f ; no functional spec for f is needed.
3. Instantiate the second query's ghost parameter with before.
4. Its postcondition gives $\text{before} \leq \text{after}$.
5. Over integers: $0 \leq (\text{after} : \mathbb{Z}) - (\text{before} : \mathbb{Z})$.

What arbitrary f can—and cannot—do

May do

- ▶ call public memory operations,
- ▶ perform any number of accesses,
- ▶ return any value or perform I/O,
- ▶ diverge.

Cannot do

- ▶ directly mutate scoped private counter state,
- ▶ forge the hidden full authority,
- ▶ make a verified public method move authority downward.

Precise claim

If f and the second query return, then before $\leq$ after. This is robustness to unknown functional behavior, not a termination proof for f .

Choose an RA by asking what must remain compatible

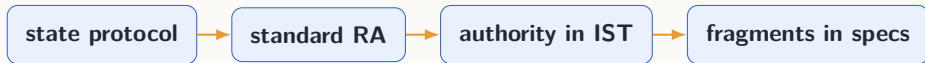
Program protocol	Useful library shape
stable agreement on a value	agree
unique, nonduplicable capability	excl
central truth plus client views	auth
independently owned keyed entries	ghost_map
increasing natural / integer	mono_nat / mono_Z
append-only history; prefix snapshots	mono_list

The RA operation and validity relation implement the protocol.

Upstream Iris provides these algebra libraries. CRIS already ports `ghost_map` and `mono_list`; a matching local `mono_nat` wrapper remains to be added.

A four-question design recipe

1. **Private state:** what concrete state must the module protect?
2. **Client view:** what capability or durable fact should cross the API boundary?
3. **Compatibility:** which simultaneous client claims may coexist?
4. **Evolution:** which updates must preserve all compatible frames?



Composition

A module may enforce several independent protocols at once by combining their resources with a product / separating conjunction.

Three rules to take to the afternoon

1. Exact lookup

$$\text{Auth}_{\gamma_m}(M) * \ell \mapsto_{\gamma_m}^q v \implies M(\ell) = v$$

2. Permission accounting

full element $\Rightarrow$ update/free fraction $\Rightarrow$ read

3. Past versus current

$$\text{Auth}_{\gamma_c}(n) * \text{Snap}_{\gamma_c}(m) \implies m \leq n$$

Resources are executable proof protocols for modular clients.

Backup: current artifact vs. ghost_map

Current imp_system/mem	Standard-library presentation
nested discrete functions + optionUR + dfrac_agreeR	ghost_mapG hides gmap_viewR
own mem_name (âUR mem_src)	ghost_map_auth gamma 1 M
mem_points_to_singleton	ghost_map_elem gamma 1 dq v
mem_ra_lookup	ghost_map_lookup
mem_ra_update	ghost_map_update
mem_ra_free	ghost_map_delete
mem_ra_alloc	ghost_map_insert(_big)

Why teach the library version?

It exposes the reusable protocol and avoids making the first encounter depend on implementation details of the camera carrier.

Backup: exact installed ghost_map API

```
ghost_map_auth gamma q m
ghost_map_elem gamma k (DfracOwn q) v
ghost_map_elem gamma k (DfracOwn 1) v
ghost_map_elem gamma k DfracDiscarded v

ghost_map_lookup
ghost_map_insert
ghost_map_update
ghost_map_delete
ghost_map_elem_persist
```

Source: `_opam/lib/coq/user-contrib/iris/base_logic/lib/ghost_map.v`. Definitions/notation: lines 28–49; fractions/persistence: 62–69 and 136–150; lookup/update rules: 221–279. CRIS's main compatibility port is at `CRIS/theories/iris_system/lib/ghost_map.v`.

Backup: the checked-in core memory specs

```
alloc:  ret = Vptr (b,0)
        * (b,0) |-> replicate sz Vundef

load:   (b,ofs) |->{q} v
        ==> ret = v * (b,ofs) |->{q} v

store:  (b,ofs) |-> v_old
        ==> ret = 0 * (b,ofs) |-> v_new

free:   (b,ofs) |-> v
        ==> ret = 0
```

Source: `imp_system/mem/MemA.v:220-239`. The implementation also exports `compare` and `CAS`; omit them from the first explanation.

Backup: exact installed mono_nat API

```
mono_nat_auth_own gamma q n      (* exact authority *)
mono_nat_lb_own gamma m          (* persistent lower bound *)

mono_nat_lb_own_get              (* auth n -> lower bound n *)
mono_nat_own_update              (* n <= n' -> update auth *)
mono_nat_lb_own_valid            (* auth n + lb m -> m <= n *)
```

Source: `_opam/lib/coq/user-contrib/iris/base_logic/lib/mono_nat.v`. The header uses stale shorthand names in prose; the identifiers above are the actual definitions in this checkout. They require a small adapter to CRIS's non-step-indexed `own` before use in a CRIS IST.

Backup: access-counter engineering choices

choice	consequence
Does query count?	Monotonicity survives either choice; benchmark overhead and interpretation differ.
What counts as access?	Fix alloc/load/store/free, or just load/store, before writing the specs.
What about CAS?	Count one public CAS or count its internal load/store calls—but state which.
Machine overflow?	Exclude it, saturate, use unbounded runtime integers, or weaken the abstraction to modular behavior.
What if $\mathfrak{f}$ diverges?	No second query; the post-return inequality is not a termination claim.

Backup: DFrac has a third internal shape

- ▶ `DfracOwn q`: own quantitative fraction q .
- ▶ `DfracDiscarded`: know that a fraction was discarded; persistent.
- ▶ `DfracBoth q`: own q and know some fraction was discarded.

$$\text{DfracBoth}(q) = \text{DfracOwn}(q) \cdot \text{DfracDiscarded}$$

Why it matters

Once discarded knowledge exists, validity requires owned fractions to total strictly less than 1. Therefore nobody can hold full write authority now or in the future.

Source: `iris/algebra/dfrac.v:1-20, 26-106`.

Backup: the paper's hybrid memory is a different point

Section 7 / Figure 12

The paper's hybrid specification uses take so a client proof may select either:

- ▶ an ownership-based memory specification, or
- ▶ a concrete operational memory model with nondeterministic allocation.

Do not conflate the examples

Today's `ghost_map` story asks how to implement the ownership protocol cleanly. The paper's example asks how an imaginary specification can offer ownership and operational reasoning as alternative branches.

Backup: source anchors for the mechanized store proof

- ▶ Concrete SGet/SPut store: `MemI.v:111-117`.
- ▶ Public full-ownership store spec: `MemA.v:235-238`.
- ▶ Logical/concrete heap relation: `MemIproof.v:9-13`.
- ▶ IST with full authoritative ownership: `MemIproof.v:224-228`.
- ▶ Lookup lemma: `MemIproof.v:116-138`.
- ▶ Joint authority/fragment update: `MemIproof.v:140-162`.
- ▶ Store simulation: `MemIproof.v:325-346`.

Reading strategy

Align the proof script with the five conceptual steps on Slide 16. Treat tactics as bookkeeping around the same resource transition.